

OpenCode 原生 Loop 改造

可行性与架构方案

参考 Cursor /loop 与 Ralph Loop，对 OpenCode v1.18.16 / dev / V2 Runtime 的源码级调查

调查日期：2026-08-12

结论类型：架构评估 + 改造设计 + 实施路径

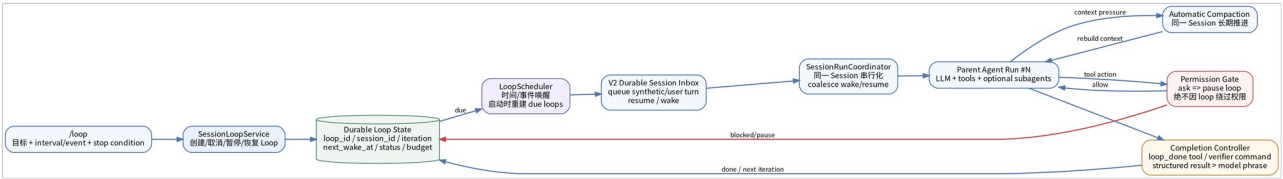


图1 建议的 OpenCode 原生 Loop 总体架构 (本报告方案)

0. 结论先给

最终结论 能。Loop 完全可以引入新版本 OpenCode，而且 OpenCode 当前 V2 架构已经具备实现它的大部分底座。推荐把它做成 Session Runtime 的一等能力，而不是一个外层 while、Shell wrapper 或纯 Skill。

一句话判断：如果目标只是“有一个 /loop 命令”，插件层即可做出可用 PoC；如果目标是达到 Cursor 那种“同一 Session、多次 Agent Run、可 sleep/wake、上下文自动压缩、可取消且不重复执行”的长期 Agent 体验，则需要在 OpenCode V2 Session Runtime 中加入持久化 Loop Controller / Scheduler。

问题	结论	置信度	说明
Loop 能否加入 OpenCode?	能	高	现有 Session、Runner、Compaction、Command、Skill、Plugin、V2 durable inbox 已提供主要原语。
能否只靠一个 Skill 完成?	不能做到生产级	高	Skill 只能教模型“怎么循环”，不能保证调度、持久化、并发串行化、取消和崩溃恢复。
能否只靠 Plugin 完成?	能做 PoC，难保证原生级可靠性	高	V2 plugin 可 prompt/wait/订阅事件并维护 timer，但缺少足够稳定的 terminal-turn / durable scheduler 契约。
是否应该复用现有 SessionPrompt.loop()?	不要直接把它当产品 Loop	高	它是单次用户 turn 内的 agent/tool continuation loop，不等价于跨 turn 的 schedule/wakeup loop。
是否应该基于 V2?	是	高	V2 已定义 durable inbox、queue/steer、wake/resume、SessionRunCoordinator 和自动 compaction。
Loop 是否默认新建 Subagent?	不应	高	默认应在同一 Parent Session 上新增 Agent Run；Subagent 仍由 task 工具按需创建。

为什么能

- OpenCode 已有 Session 级长期容器、持久化消息历史和自动 compaction，因此“同一会话跑很多轮”的基础已经存在。

- 当前源码的 SessionPrompt 接口本身就暴露 internal loop(); 这说明 OpenCode 的 agent 执行已经以 loop/state-machine 方式组织，而非单次 RPC。
- V2 Session 规范已经把“录入输入”和“执行输入”拆开：Prompt 先进入 durable inbox，再由 runner 在安全边界 promote；这正适合 scheduler 产生下一轮输入。
- V2 明确设计 SessionRunCoordinator：同一 Session 的 resume 会 join，wake 会 coalesce，不同 Session 可并行。这正是长期 Loop 最需要的串行化原语。
- V2 API 已有 prompt、command、wait、pending、interrupt、compact、context 等接口；Loop 不需要重新发明 agent 本身。
- OpenCode 的 Command / Skill / Plugin 表面层足以承载 /loop UX；真正新增的主要是持久化 Loop 状态与唤醒逻辑。

为什么又不能“直接加一个循环就完事”

- V1 的 SessionRunState 仍依赖进程内 Map<SessionID, Runner>；历史 issue 已出现同一 Session 重复 provider turn、异步 prompt 重入、busy 状态滞后等现象。长期 Loop 会放大这些问题。
- Cursor 的关键不是 /loop 这个名字，而是 Runtime 有明确 stop/follow-up 边界、loop_count、scheduler 与 session 级生命周期。OpenCode 目前没有完全对等的稳定 stop-followup 契约。
- 纯插件 timer 在进程退出、插件 reload、Desktop/Server 重启后会丢；如果 Loop 状态不持久化，就不是“原生长期 Agent”。
- V2 规范自己明确写着：post-crash continuation recovery、clustered Session execution ownership 等仍是后续项。因此跨崩溃“自动重放未确定 provider turn”现在不能安全假设。
- 固定 interval 很容易；“Agent 决定什么事件唤醒自己”需要事件注册、predicate、外部 watcher/backoff，以及防止事件风暴的 coalescing。
- 长期 unattended loop 与权限系统存在天然冲突：不能为了循环顺畅就自动绕过 ask/deny。遇到权限请求时必须暂停或进入 blocked 状态。

1. 调查范围与版本基线

本报告以 2026-08-12 可访问的官方资料和公开源码为基线。OpenCode 官方 Changelog 最新稳定版本为 v1.18.16 (2026-08-10)；同时检查 anomalyco/opencode 的 dev 分支和 V2 Session 规范。Cursor 侧检查 3.5 的 /loop 发布说明、Hooks 文档与官方 cursor/plugins 中的 ralph-loop。

对象	本次使用的权威材料	关键用途
OpenCode Stable	Changelog v1.18.16	确认当前稳定版本与 V2 迁移背景
OpenCode V1/dev	session/prompt.ts, run-state.ts, compaction.ts	确认当前内部 agent loop、runner、compaction
OpenCode V2	specs/v2/session.md, V2 API / plugin docs	确认 durable inbox、wake/resume、coordinator、插件能力

对象	本次使用的权威材料	关键用途
Cursor /loop	Cursor 3.5 changelog	确认产品语义：local schedule、until outcome、agent-decided wakeup
Cursor Hooks	stop/followup_message/loop_count	确认 Runtime continuation primitive
Cursor Ralph	hooks.json, stop-hook.sh	确认 stop -> followup 以及外置 loop state 的真实实现

2. Cursor 值得借鉴的不是 /loop 命令，而是 Runtime 语义

Cursor 3.5 将 /loop 定义为“在本地 schedule 上重复运行 prompt，直到目标达成或用户停止；没有固定 interval 时，Agent 可决定何时或由什么事件唤醒”。这个定义包含四个不可省略的语义：Session 延续、Run 分段、Wakeup、Stop。

- 1. Session 延续：逻辑上是同一 conversation，而不是每轮创建完全独立的新聊天。
- 2. Run 分段：每次 wake 后是新一轮 Agent turn / generation，不是一个 LLM 请求永久挂起。
- 3. Wakeup：可以是时间，也可以是事件；sleep 时不需要持续占用推理。
- 4. Stop：有明确终止边界，能够由 runtime/验证器决定是否继续。

最重要的 Cursor 原语 stop hook 可以返回 followup_message，Cursor 自动把它作为该 conversation 的下一条 user message；loop_count 由 Runtime 维护，loop_limit 可配置。

Cursor 官方 ralph-loop 进一步证明：其 stop-hook.sh 并不重新启动 Cursor 进程，而是在 agent turn 结束后读取持久化 scratchpad，更新 iteration，再输出 followup_message。也就是说，真正的“原生”不是 while(true)，而是 Session Runtime 承担 continuation。

3. OpenCode 当前实际上已经有两种“loop”，但都不是 Cursor /loop 产品语义

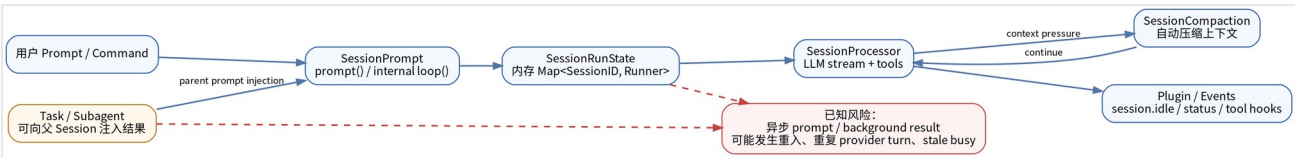


图2 OpenCode 当前相关执行链：已有 agent loop，但长期 scheduler 仍缺位

3.1 SessionPrompt.loop()：单个用户任务的内部 Agent Loop

当前 dev 源码的 SessionPrompt.Interface 直接包含 prompt() 与 loop()。这里的 loop 解决的是一个用户 turn 内“LLM -> tool calls -> tool results -> 下一次 provider turn -> finish”的持续执行，而不是“结束后睡 5 分钟，再把 prompt 作为下一条消息运行”。

不要混淆 SessionPrompt.loop() 是 agent execution loop；Cursor /loop 是跨 turn 的 long-running schedule loop。名字相同，抽象层不同。

3.2 SessionRunState：V1 试图做到“一 Session 一 Runner”

run-state.ts 中 SessionRunState 使用 InstanceState 内的 Map<SessionID, Runner>；ensureRunning() 复用同一 session 的 runner，并在 onBusy/onIdle 更新状态。这个设计意图非常适合作为 Loop 基础，但它是进程内状态，并非持久化执行所有权。

3.3 Compaction：长期同 Session 的上下文基础已经存在

OpenCode 已有 SessionCompaction，并支持 repeated compaction。V2 规范更进一步：完整 transcript 保持 durable，模型可见表示换成 rolling summary + token-bounded recent context；下一 provider attempt 再基于新的 context epoch 重建。Loop 因此不需要扩展模型 context window，只需要复用 compaction。

3.4 V2 才是最关键的“原生 Loop 地基”

V2 Session 规范已经从“HTTP 调 prompt 然后立即执行”升级为 durable admission + serialized runner：

```
sessions.prompt(...)
  -> durable Session inbox
  -> optional resume/wake
  -> SessionRunCoordinator
  -> promote eligible input at safe boundary
  -> one provider turn at a time
  -> tool settlement
  -> continuation / idle
```

其中 queue 输入在当前 drain 需要 continuation 时保持 FIFO，只有 Session 本来要 idle 时才 promote；steer 则在安全 provider-turn boundary 注入。这意味着 Loop 下一轮最自然的实现就是“向同一 Session 的 durable inbox 写一个 queue input，然后 wake/resume”。

4. 为什么 OpenCode 能做成 Cursor 风格 Loop

4.1 Session 是现成的长期容器

Loop 不需要自己维护整段聊天记录。sessionID、message history、agent/model selection、permission、compaction 都已有。

4.2 已有明确的“执行 vs 输入”分离

V2 prompt 的 admission 与 execution 解耦，使得 scheduler 不需要直接调用 LLM；它只需要持久化下一次输入并发 wake。

4.3 已有同 Session 串行化设计

SessionRunCoordinator 的目标就是同一 Session 不重入、不同 Session 可并行；Loop 只应在这个协调器上排队，不能旁路。

4.4 已有等待、取消和 pending API

/wait、/interrupt、/pending、/prompt 等接口能够支持 UI、插件与测试 harness 观察 Loop。

4.5 已有 Skill / Command / Plugin 表面层

可以用 /loop 做用户入口、Skill 做 prompt 规范、Plugin 做实验，但核心 schedule 不必耦合 UI。

4.6 已有 Subagent 模型

Loop 应继续驱动 Parent Session；Task/Subagent 仍只在某个 iteration 内按需使用，避免把“循环”错误等同于“每轮新 Agent”。

5. 为什么不能只复制 Cursor 的表面实现

5.1 V1 并发不变量还不够稳

公开 issue #31487 给出了同一 session 中两个真实 provider stream 重叠的日志，并指出 background task result injection 可能导致两个 prompt loop 交错。issue #30809 还展示了 loop completion 依赖消息 ID 排序而非 parent 关系时会产生重复 sibling assistant。对长 Loop 而言，这不是小 bug：每一次 wake 都会增加一次触发并发边界的机会。

5.2 session.status/idle 不能被当作唯一真相

issue #35472 说明 HTTP prompt 已完成，V1 的 status map 仍可能短暂甚至长期显示 busy。若一个插件用“poll idle -> 继续下一轮”的方式实现 /loop，会遇到误阻塞或竞态。因此生产实现应使用 runner 的 terminal boundary / durable input ownership，而不是 UI status 轮询。

5.3 Plugin 能做 timer，但不是 durable scheduler

V2 plugin setup 可以拥有 timers/watchers，也可以 ctx.session.prompt()/wait()/event.subscribe()。这足以做原型；但 plugin reload、进程退出时 timer 必须 cleanup，重启后不会自动恢复某个未来 wake。要称为原生 Loop，next_wake_at、iteration、status 必须落到持久化状态。

5.4 Crash recovery 目前仍有明确边界

V2 规范明确把 post-crash continuation recovery 与 clustered Session execution ownership 列为后续工作。因此设计 Loop 时必须区分“睡眠期间重启”与“provider turn 中途崩溃”：前者可以安全恢复 due wake；后者不能盲目重发，因为工具可能已经产生外部副作用。

5.5 权限系统是长期 Agent 的第一等约束

OpenCode 的 permission 语义是 allow / ask / deny。Loop 不应新建“自动批准所有工具”的旁路。推荐：若任一工具触发 ask，Loop 状态从 RUNNING 变为 BLOCKED，等待人工批准后再 resume；deny 则将本轮视为可观察失败，并让模型决定替代方案或达到 retry 上限后停止。

6. 推荐架构：把 Loop 做成 V2 Session Runtime 的一等能力

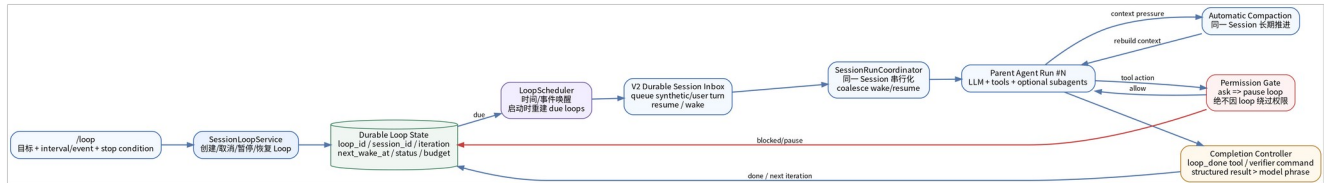


图3 推荐架构：SessionLoopService + Durable State + Scheduler + V2 Inbox/Coordinator

设计原则 Scheduler 只负责“何时产生下一条输入”；SessionRunCoordinator 负责“是否能执行”；LLM/Tools 继续由现有 Agent Runtime 负责。不要让 Loop Scheduler 直接调用 provider。

6.1 新增 SessionLoopService

建议新增一个进程级服务，接口负责 create / cancel / pause / resume / get / list，业务状态落 SQLite/Event Store，而非 Map 或项目 scratch 文件。

```
type LoopSpec = {
  sessionID: SessionID
  prompt: string
  wake: { type: "interval"; everyMs: number }
    | { type: "event"; predicate: EventPredicate }
    | { type: "adaptive"; maxSleepMs?: number }
  stop: StopPolicy
  maxIterations?: number
  maxDurationMs?: number
  maxCostUsd?: number
}

type LoopState = {
  loopID: LoopID
  sessionID: SessionID
  iteration: number
  status: "scheduled" | "running" | "blocked" | "paused" | "done" | "cancelled" | "error"
  nextWakeAt?: number
  lastInputID?: MessageID
  lastAssistantID?: MessageID
  createdAt: number
}
```

```

    updatedAt: number
  }

```

6.2 Scheduler 不直接运行 Agent，只写 durable inbox + wake

当 next_wake_at 到期，LoopScheduler 应原子地 claim 该 wake，然后通过 V2 sessions.prompt()/synthetic admission 将下一轮加入同一 session 的 queue，最后调用 wake/resume。这样如果 session 当前正在工具调用或 provider continuation，queue 会等待安全 idle boundary，而不会并发进入第二个 stream。

6.3 每轮是一条新的 Agent Run，不是复用一条永不结束的请求

```

Session C1
  User: /loop ...
  Assistant Run #1 ... finish
  [sleep / wait]
  Synthetic/User: [loop iteration 2] ...
  Assistant Run #2 ... finish
  [sleep / wait]
  ...
  [automatic compaction when needed]
  Assistant Run #N ... finish

```

这与 Cursor 的核心语义一致：同一 Parent Session 保留任务连续性，但每轮是独立 provider turn/agent run；上下文到阈值时由 compaction 保持窗口预算。

6.4 Stop 不要依赖“模型说 COMPLETE”一个信号

Cursor Ralph 使用 completion promise 是非常实用的插件技巧，但如果把 Loop 变成 OpenCode 核心能力，应把终止证据结构化。推荐优先级：

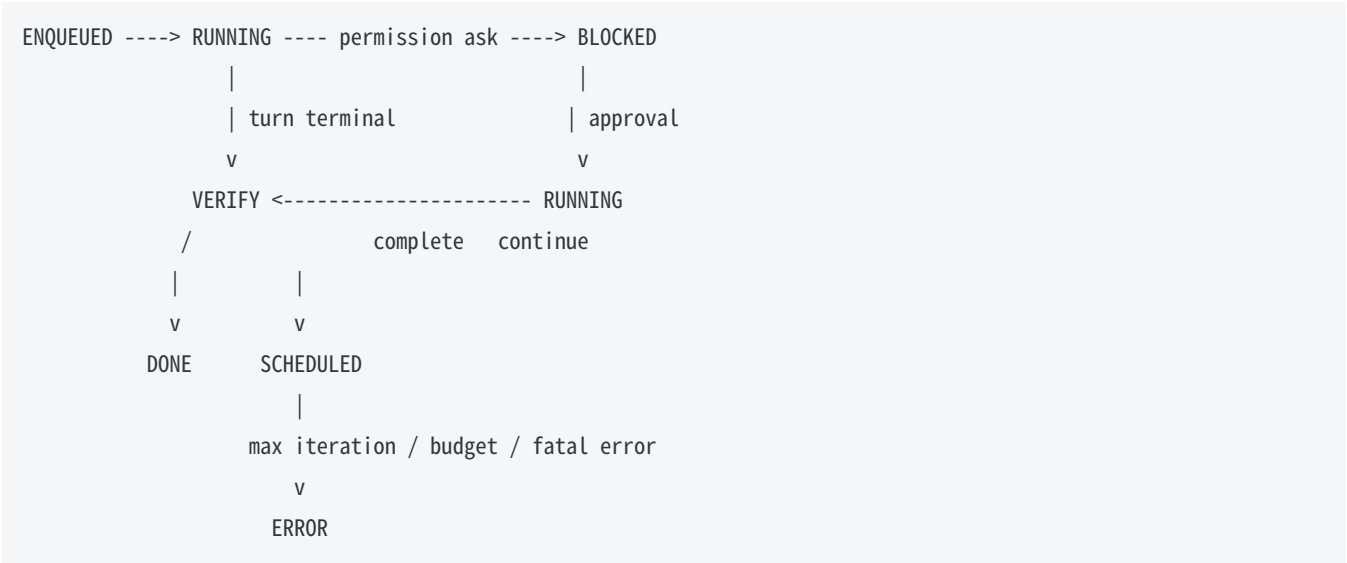
5. 确定性 verifier：测试命令、构建命令、HTTP/CI 状态或用户定义 predicate。
6. loop_done 工具：Agent 显式调用，并提交 reason/evidence；Runtime 再校验可选 verifier。
7. 结构化 final result：例如 {continue, reason, nextWake}。
8. 自然语言 completion phrase：仅作为兼容/便捷模式，不能是默认生产判据。

7. 建议的 Loop 状态机

```

CREATED
|
v
SCHEDULED ---- cancel ----> CANCELLED
|
due/event
v

```



关键不变量：一次 Loop iteration 必须绑定一个 durable input ID；只有该 input 对应的 terminal assistant 已提交，才允许 iteration++。不能用“最新 assistant 时间戳”或“status 变 idle”猜测完成。

8. 推荐源码改造点（面向 OpenCode dev / V2 ）

文件/模块	建议改造	优先级
packages/opencode/src/session/loop.ts (新增)	SessionLoopService: 创建、取消、暂停、恢复、状态机、迭代推进	P0
core/session SQL / event schema	持久化 LoopSpec/LoopState; loop_id + session_id + iteration + nextWakeAt + status	P0
V2 SessionExecution / SessionRunCoordinator	暴露安全 terminal boundary / wake completion; 禁止 scheduler 绕过 coordinator	P0
V2 session route/API	POST /session/:id/loop, GET loop, POST cancel/pause/resume	P1
Command registry	内置 /loop 与 /loop-cancel; 自然语言参数转换成 LoopSpec	P1
Skill	提供 loop 目标、验证器、停止条件写法; 不是调度器本身	P1
LoopScheduler (新增)	本地 timer wheel / min-heap; 重启扫描 due loops; wake coalescing	P1

文件/模块	建议改造	优先级
Event bridge	loop.created / woke / iteration.completed / blocked / completed / cancelled / error	P1
Tool registry	loop_done / loop_status（可选）；提供结构化完成证据	P2
Plugin V2	新增 terminal-turn/loop lifecycle hook，允许插件扩展 verifier/wakeup predicate	P2
Desktop/TUI	Session header 展示 Loop N、next wake、pause/cancel；blocked 权限状态	P2
Task tool	仅透传 loop_id/iteration 作为 metadata；不让 subagent 自己推进 parent loop	P2

8.1 不建议直接大改的模块

- LLM / Provider: Loop 不需要改变模型调用协议，只要每次 iteration 最终仍走正常 provider turn。
- Compaction: 第一版直接复用现有 automatic compaction；只有在长期测试发现 loop state 丢失时，再做 loop-aware compaction context。
- Task/Subagent: 不要把 Loop 做成“每轮 spawn subagent”；保持现有 task 语义，主 Session 负责长期任务所有权。

9. 两条实施路线：Plugin PoC 与 Core Native

能力	Plugin PoC	Core Native	建议
固定 interval	可以: timer + ctx.session.prompt/wait	可以: durable scheduler	PoC 先做
同一 Session	可以	可以	都可
自动 compaction	复用现有	复用 V2	都可
取消	状态文件 + clear timer	原子 status=cancelled + scheduler fencing	核心更可靠
进程重启后继续	需自行扫描文件恢复，语义脆弱	DB due-loop recovery	必须核心化
同 Session 串行	依赖 API/coordinator，插件需避免竞态	runtime 保证	必须核心化

能力	Plugin PoC	Core Native	建议
stop/terminal boundary	只能 event/status/wait 推断或包装	一等 terminal-turn 事件	必须核心化
event wakeup	可订阅部分 event	可建 predicate registry/coalesce	核心更完整
permission blocked	插件难做统一状态机	Runtime 可直接感知	核心化
预算/成本/迭代一致性	自行维护	统一 policy	核心化
崩溃/重复执行控制	弱	可结合 durable claim/fencing	核心化

判断 建议先做 Plugin PoC 验证 UX 和 prompt 行为，但不要把 PoC 直接当生产实现。生产版应把 state + schedule + terminal boundary 下沉到 V2 Core。

10. 可快速验证的 Plugin PoC 方案

PoC 的目的不是证明“timer 能工作”，而是验证：同一 Session 的多轮执行是否提升 coding task 完成率、compaction 是否可承受、用户是否理解 pause/cancel，以及模型是否能稳定使用 verifier。

- 9. 新增 /loop command 或 loop skill，解析目标、interval、maxIterations、stop condition。
- 10. 把 LoopState 写到 .opencode/loops/<loopID>.json（仅 PoC；生产改 DB）。
- 11. Plugin setup 启动 timer；每个 due wake 先 ctx.session.wait(sessionID)，然后 ctx.session.prompt() 同一 session。
- 12. 每次 prompt 前写 iteration claim，prompt 完成后读取结果/运行 verifier，再决定 schedule next。
- 13. /loop-cancel 将 state 置 cancelled，并清 timer；不要只删除文件，以避免并发 callback 已启动。
- 14. 若 permission/question 阻塞，不要继续 schedule；记录 blocked，等待 session event 后再恢复。

PoC 限制 不要宣称 crash-safe；不要使用 V1 session.status 作为唯一完成信号；不要允许同一个 session 同时有两个 loop。

11. Core Native 的核心伪代码

```
async function wakeLoop(loopID) {
  const loop = await loopStore.claimDue(loopID)
  if (!loop || loop.status !== "scheduled") return

  // 1. 不直接 call LLM，先进入同一 Session 的 durable inbox
  const input = await sessions.prompt({
    sessionID: loop.sessionID,
    text: renderIterationPrompt(loop),
```

```
    delivery: "queue",
    resume: false,
    metadata: { loopID, iteration: loop.iteration + 1 }
  })

  // 2. 写入 durable 绑定关系，之后再 wake
  await loopStore.markEnqueued(loopID, input.id)
  await sessionExecution.wake(loop.sessionID)
}

async function onTerminalTurn(event) {
  const loop = await loopStore.byInput(event.parentInputID)
  if (!loop) return

  const verdict = await verifier.evaluate(loop, event)
  if (verdict.complete) return loopStore.finish(loop.id, verdict)
  if (budgetExceeded(loop)) return loopStore.fail(loop.id, "budget")

  const next = await wakePolicy.next(loop, event, verdict)
  await loopStore.schedule(loop.id, next)
}
```

两个关键点：先持久化 admission 再 wake；下一轮 schedule 只能由明确的 terminal-turn 事件触发。这能避免“wake 已发生但 state 还没写”的重复执行窗口。

12. 同一 Session 与上下文策略

推荐默认与 Cursor 一样保持同一 Parent Session。原因是 coding loop 往往需要保留用户目标、已做决定、工具结果与最近验证失败；每轮新建独立 session 会增加 handoff 损失。

策略	优点	缺点	推荐场景
Same-session + compaction	连续性最好；复用现有历史、权限、模型	长时间后 summary 会损失细节	默认
Fresh child session per iteration	每轮 context 干净，适合隔离	必须外置状态/总结，handoff 容易失真	非常长的批处理
Hybrid checkpoint	每 N 轮建立 checkpoint/fork，再继续	实现更复杂	100+ 轮长期任务

无论选择哪种模式，关键任务状态都应外置：测试结果、feature checklist、git commit、acceptance artifact、LoopState 不应只依赖 conversation summary。

13. Wakeup 设计：先固定间隔，再做事件与自适应

模式	实现	复杂度	第一版?
interval	nextWakeAt = now + everyMs; 持久化后由 timer/min-heap 唤醒	低	是
backoff	失败时指数退避，成功/变化时缩短	低-中	是
file/event	订阅 file.watcher.updated、session/tool/外部 adapter 事件，predicate match 后 wake	中	第二版
adaptive agent-decided	Agent 输出 next_wake_at / event predicate，Runtime 校验边界与最大等待	中-高	第二/三版
external webhook	需要服务器持久路由、鉴权、去重与映射 loopID	高	后续

Cursor “Agent decides when or what event should wake it” 值得参考，但不建议第一版直接开放任意事件表达式。先定义受控 predicate 类型（file changed / command state / HTTP status / session event），再逐渐扩展。

14. 权限、资源与成本控制

长期 Loop 的危险不是“会不会继续”，而是“继续多少次、花多少 token、做多少有副作用的工具操作”。因此 LoopSpec 必须内置预算，不应只依赖 prompt。

- maxIterations：默认有限值，例如 20；用户可显式提高。
- maxDuration：限制 wall-clock 生命周期，避免忘记的 loop 永久存在。
- maxCost / maxTokens：模型层预算；达到阈值后进入 paused-budget。
- tool policy：继续使用 Agent/Session permission；Loop 不额外提升权限。
- question/permission：需要用户时状态 BLOCKED，并展示原因，不继续自动 wake。
- rate limit：同一 Session 单 Loop；同项目并发 Loop 数可配；event wake 做 debounce/coalesce。
- side-effect fencing：每个 iteration 有唯一 input/claim id，外部 verifier 或 tool 可携带 idempotency key。

15. 故障模型与恢复策略

故障点	是否可自动恢复	推荐行为
sleep/scheduled 时进程退出	是	重启扫描 status=scheduled 且 nextWakeAt<=now；原子 claim 后 wake

故障点	是否可自动恢复	推荐行为
已 enqueue 但未 wake 时退出	是	durable inbox 仍在；重启 resume/wake session
provider 尚未产生 durable assistant/tool 时异常	谨慎	可在明确 provider-not-dispatched 证据下 retry；否则需人工/显式 run
tool 已产生外部副作用后崩溃	不能盲重放	依赖 tool settlement、idempotency 或人工确认
permission ask	不应自动恢复为 allow	BLOCKED，等待用户
verifier 超时	可	按 retry policy 重试 verifier，不重新执行 coding turn
compaction 失败	有限	保留上一有效 history boundary，停止/重试 compaction；不要丢原任务

16. 必须先修/先验证的 OpenCode 现有问题

问题族	对 Loop 的影响	上线前要求
#31487 / #30809 同 Session 重叠/重复 turn	重复成本、重复编辑、历史污染	基于 V2 coordinator；同 session 任意时刻最多一个 provider stream；parent/input 绑定完成判定
#35472 stale busy	插件用 status 驱动时可能错误等待	Loop 不依赖 status map 作为真相；用 durable terminal event
compaction 历史问题	长期 Loop 信息丢失或 summary 异常	100-loop soak 覆盖多次 compaction；关键状态外置
subagent hang/timeout 类问题	父 Loop 可能永不进入 terminal	Task 必须有可观察终止/interrupt；Loop 预算包含子任务
provider/network retry	单轮可能卡住	沿用 provider retry + 增加 Loop iteration watchdog，不在 scheduler 中重复发 prompt

17. 测试与验收：什么才叫“Loop 做成了”

17.1 Runtime 不变量测试

- 同一 session 在任何时刻最多一个 llm.stream/provider turn。

- 一个 loop iteration 只对应一个 durable input；一个 input 只推进 iteration 一次。
- 重复 wake / timer callback / event storm 不会产生重复 prompt。
- cancel 后即使已有 timer callback 排队，也不能再 enqueue 下一轮。
- session busy 时 loop input 必须进入 queue，而不是并发执行。
- permission ask 会 BLOCKED，批准后只恢复一次。
- compaction 前后 loopID、iteration、goal/verifier 不丢。

17.2 100 轮 Soak Test

建议用确定性 fake provider + 真实文件工具做 100 轮测试，不要只用模型主观评估。至少覆盖：每 3 轮一次 tool call、每 10 轮触发 compaction、随机插入网络 retry、随机重启 runtime、随机 permission block、随机取消。

指标	目标
duplicate provider turns	0
duplicate iteration admission	0
lost iterations after restart	0（仅安全边界）
cancel leakage	0
session history corruption	0
context overflow unrecovered	0（在可 compaction 情况下）
blocked permission auto-bypass	0
loop state/message binding mismatch	0

17.3 能力评测：不要把“跑得久”误认为“能力更强”

Loop 是 harness 能力，不会自动提升底层模型智力。真正比较 Cursor / Claude Code / OpenCode 时应分开测：单轮 coding 能力、长任务恢复能力、context degradation、工具可靠性、验证闭环、成本与重复工作率。

18. 推荐实施顺序

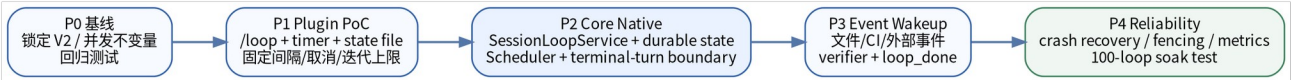


图 4 推荐实施阶段

阶段	产物	Go/No-Go 条件
P0 基线	锁定 V2 分支；补并发/parent-boundary 回归测试；观测 provider stream count	同 Session 重入测试为 0
P1 Plugin PoC	/loop、/loop-cancel、固定 interval、max iterations、state file、verifier	20 轮稳定；无重复 prompt；用户可取消
P2 Core Native	SessionLoopService、DB state、scheduler、terminal event、V2 inbox integration	重启恢复 sleeping loop；100 轮基础 soak
P3 Event/Adaptive	受控 event predicate、backoff、loop_done、结构化 verifier	事件风暴不重复执行；blocked 权限正确
P4 Reliability	crash fencing、metrics、budget、UI、fault injection	100 轮 fault soak 通过；成本/重复率达标

19. 建议的用户体验与 API（提案，不是现有 OpenCode 语法）

```
/loop 每 5 分钟检查一次测试状态；如果测试失败就继续修复，直到：
- unit tests 全部通过
- lint 通过
- typecheck 通过
最多 20 轮；需要权限时暂停并通知我。

/loop-status
Loop lp_xxx  RUNNING
session: ses_xxx
iteration: 7 / 20
next wake: 4m32s
verifier: npm test && npm run lint && npm run typecheck
cost: $1.84 / $5.00

/loop-pause
/loop-resume
/loop-cancel
```

API 可以对应为：POST /api/session/{sessionID}/loop、GET /loop/{loopID}、POST /pause、/resume、/cancel；另外在 session events 中发布 loop.* 状态。

20. 最终工程建议

推荐决策 做，而且从 V2 做。把 /loop 定义成“Session 上的持久化调度能力”，而不是“Prompt 自己循

环”。

最合理的产品/工程分层是：

- 15. /loop Command：用户入口，负责把自然语言变成 LoopSpec。
- 16. Loop Skill：教 Agent 如何写 stop condition、verifier、next wake；不拥有调度。
- 17. SessionLoopService：持久状态机与政策。
- 18. LoopScheduler：时间/事件唤醒。
- 19. V2 durable inbox + SessionRunCoordinator：真正执行下一轮并保证 session 串行。
- 20. Compaction：维持同一 Session 的有限 context window。
- 21. Verifier / loop_done：决定是否结束，而不是靠自然语言猜。

如果只能做一个最关键的核心改动：先补一个“durable terminal-turn boundary + follow-up admission”原语。它相当于 Cursor stop -> followup_message 的 OpenCode 版本。Scheduler、Ralph、重试、CI polling、持续测试等能力都能建立在这个原语上。

不建议 不要把 V1 SessionPrompt.loop() 直接暴露成 /loop；不要在插件里轮询 session.status 后 prompt_async；不要在每轮 spawn 新 Session/Subagent；不要默认无限循环；不要让 Loop 绕过 permission。

附录 A：关键证据与来源

编号	来源	链接
S1	OpenCode Changelog - v1.18.16, Aug 10 2026	打开来源
S2	OpenCode dev - packages/opencode/src/session/prompt.ts	打开来源
S3	OpenCode dev - packages/opencode/src/session/run-state.ts	打开来源
S4	OpenCode dev - packages/opencode/src/session/compaction.ts	打开来源
S5	OpenCode V2 Session API/Runtime specification	打开来源
S6	OpenCode V2 Plugin API	打开来源

编号	来源	链接
S7	OpenCode V2 API - Send message / durable admission	打开来源
S8	OpenCode V2 API overview	打开来源
S9	OpenCode Plugins / events	打开来源
S10	OpenCode Agent Skills	打开来源
S11	OpenCode Commands	打开来源
S12	OpenCode Issue #31487 - overlapping LLM streams in one session	打开来源
S13	OpenCode Issue #30809 - duplicate same-parent assistant replies	打开来源
S14	OpenCode Issue #35472 - stale busy session status	打开来源
S15	Cursor 3.5 - Shared Canvases and /loop Skill	打开来源
S16	Cursor Hooks - stop / followup_message / loop_count	打开来源
S17	Cursor official ralph-loop hooks.json	打开来源
S18	Cursor official ralph-loop stop-hook.sh	打开来源
S19	Cursor official ralph-loop Skill	打开来源

附录 B：结论的证据链

结论	主要证据
OpenCode 已有内部 agent loop	S2: SessionPrompt.Interface 直接包含 loop(input: LoopInput)。
V1 runner 不是 durable ownership	S3: Runner 存在 InstanceState 的内存 Map<SessionID, Runner>。
同 Session 长期上下文可通过 compaction	S4/S5: 已有自动/重复 compaction; V2 用 rolling summary + recent context。

结论	主要证据
V2 适合做 Loop Scheduler	S5/S7: durable inbox、resume/wake、queue/steer、SessionRunCoordinator。
纯 V1/async 实现存在并发风险	S12/S13: 真实重复/重叠 provider turn 与 completion race。
不能用 status polling 当完成真相	S14: prompt 完成后 status 仍可能 busy。
Cursor 的核心是 stop-followup continuation	S16/S17/S18: stop hook 返回 followup_message; loop_limit/null; Ralph 直接使用它。
Cursor /loop 还包含 schedule/event wakeup	S15: local schedule, 固定 interval 可省略, 由 Agent 决定 wake。

— 报告结束 —